

BLACKJACK (9-13)

#Easy #Individual #Expectation_theory



Part 0. Before you begin

Required materials:

- a sample solution file to be demonstrated to students
- A4 sheets (one sheet per student)
- Optionally: Printed cards to demonstrate game rules.

Teaching materials:

- ❑ Link to the [sample solution](#).
- ❑ Link to [game cards](#).

Recommended implementation time for the project:

Two lessons.



Part 1. General project information

Minimal prior knowledge required from students

- Each student :
- knows how to create sprites of any type in Play;
 - understands that additional files used in a project must be added to the project folder;
 - can create lists with different types of data (e.g., a sprite list)
 - can add elements to a list with `list.append()`
 - can use a construct comprised of a for list and a `range(number)` function
 - can generate random numbers (`connect random` and use `randint(a, b)`)
 - can use `async...await` construct to pause the game.

Blackjack

Learning results

- Student learned/reviewed the concept of global variables and used them in his/her program
- Student applied his/her knowledge of lists in practice (used lists to store sprites).
- Student added hints and buttons to the draft (possibly with teacher's help).
- Student programmed a user vs. computer game (where user makes a move first, and computer counters).

Ideas:

- A student became interested in thinking through computer activity against user and wants to create a more difficult game (make computer "even smarter").



Part 2. Description of the expected results

Program	Features
Blackjack	• Program welcomes a user at startup. At the bottom of the screen, two buttons are displayed: "Add card" and "Challenge computer". Program also displays user and computer scores (both are equal to zero). Nothing happens. • When "Add card" button is pressed, a random card is dealt. It is displayed to the user, and its score adds up to the user's score counter. You can add cards any number of times. • When "Challenge computer" button is pressed, a computer simulation of the game starts. Computer starts to "draw" as many cards as user added. Card scores add up to the computer's score counter. • User wins if his/her final score is greater than computer's. User loses if his/her score exceeds 21. The same applies to computer. • <i>Optional:</i> background music starts to play at the game startup. • <i>Optional:</i> player can not draw cards already drawn from the deck (real deck simulation).



Part 3. Project stages

This project will be useful to students needing a comprehensive material review. Even recent topics, such as lists and for loop, are used without complications. This project should be suggested to the students who did not implement the "gaming machine".

The project is quite easy to implement, so it is advisable to any level students.

No.	Stages and their goals	Content	Format	Results
1	"Projects fair"	Each student can select a project to work on. Then go	Work with a teacher.	Each student has

	Select from the list of available projects Get a functional specification (FS). Review the requirements and take on the project	to the platform and review functional specifications for the projects ("Blackjack: FS" or "Egg catcher: FS"). After the review, students should get started with the first task — create a project checklist.		picked up a project. Each student has received and reviewed an FS, and is ready to get started.
--	---	---	--	---

The description provided below only relates to Blackjack project stages.

2	Building a project mind map Sign in to the platform. Locate the second level of "Blackjack: FS: assignment (mind map requirements are provided at the second level) Assess whether familiar tools and theory are sufficient to complete the project (yes) Build a project mind map on your own or with teacher's support	Arrange mind map building. Remind mind map structure to students (core, outgoing beams containing blocks which also divide into smaller blocks). The purpose of a student is to describe architecture of a future program. Recommend students to open "Blackjack: FS" assignment and locate the slide containing project mind map requirements. If a student finds it hard to start building a mind map, recap Blackjack game rules and draw the beginning of the map together with him/her (the program has two parts: user gameplay and computer gameplay):  <pre> graph LR A[User's move] --> B[Blackjack] B --> C[Computer's move] </pre> A possible mind map is provided in sample project	Individual work, work with a teacher	Student created a mind map. The map describes how each sprite of the program works: score counters, buttons, cards, hints, etc. (changing counter values, showing/hiding sprites, etc.) <i>*A possible project mind map is provided in part 4 "Sample project implementation".</i>
---	--	---	---	---

		implementation.		
		Remember that the harder it is for the student to program, the more detailed the mind map needs to be.		

3	Creating UI elements Program first startup Open the project draft prog_21.py file Create button sprites for UI elements Create win(), lose(), and draw() functions to visually design victory, defeat and draw Launch the project, appreciate its appearance, correct errors	Arrange individual work. Offer student to open VSC, enter his/her login and password for mars.algoritmika.org and open "Blackjack: VSC assignment. Student must start implementing the first part of the program: making a user's move. Point out to student that he/she starts working in prog_21.py file containing a draft for his/her project. First, missing sprites need to be added to the program: Two buttons — "Add card" and "Challenge computer". Option 1. Draw the buttons (e.g. in Plaint). Then store each one in .png format and add them to the project folder via platform using "Load" button. Now you can create a button as a sprite image. Option 2. Create a sprite block and a sprite text. Block will serve as a core of the button (click will be handled particularly for this sprite) with a caption laid over it. Sprites for the option 2 may be specified as follows: <pre># "Challenge computer" button button_stop = play.new_box(color='red', border_width=1, border_color='grey', x=130, y=-160, width=190, height=40)</pre>	Individual work, work in pairs. * Creating sprites should be fairly easy, so stick to individual work as much as possible.	Blackjack containing all required sprites yet without active buttons.
---	---	--	--	--

```
stop_txt = play.new_text(words='Start me VS comp',  
font_size=30, x=130, y=-160, color='grey')
```

Now you need to create functions which will be called when the program detects that user has won or lost. They will expressly display the winner to the user.

- ❑ Let's create a text sprite named "result" which will say whether you have won or lost.

```
result = play.new_text(words='', x=-200, y=-230)
```

- ❑ Let's create lose() function indicating that user has lost. We shall also use you_score and comp_score off-the-shelf sprites from the draft.

```
def lose():  
    comp_score.color='green'  
    you_score.color='red'  
    result.color = 'red'  
    result.words = 'You lose: ' # defeat
```

Other designer functions are written in a similar fashion. A complete solution is provided at the end of methodological guidelines.

The necessary theory for the implementation of the next stage should be known to the student, but help of a teacher is required for its application.

4	Create a set of cards with numbers Think how to store all cards Think how to generate a random card	Arrange work in pairs (but do not insist if a student wants to implement this stage entirely on his/her own). Offer students to think in pairs how to generate random cards. Actually, it is sufficient to create a single image sprite. Let it originally show the first card: <pre># card: card = play.new_image(image='card1.png', size=40, x=-150, y=20) card.hide()</pre> When a random card needs to be displayed, a random number in range 1-11 is generated, and sprite image (image field responsible for image file) changes. <pre>number=randint(1,11) # pick a random number # show a card with this number and add this score to the counter: card.image = 'card' + str(number) + '.png' card.show()</pre> There is no need to waste memory on a list of cards; you can have a single sprite and change the name of the file it addresses. Note that cards are hidden by default.	Individual work, work in pairs.	A student created an image sprite for a card. Student realized how a random card from a set will be displayed.
---	---	---	--	--

5	Handling game events In @button_add.when_clicked and @button_stop.when_clicked sections (as per button names), handle mouse clicks for user's move and computer's move.	Step 1. "Add card" button press (button_add). Random card must be drawn, displayed for a short time, and hidden again. <pre>number=randint(1,11) # pick a random number # show a card with this number and add this score to the counter: card.image = 'card' + str(number) + '.png' card.show()</pre> Card score must be added to the user score counter. In steps_txt2 off-the-shelf sprite in the draft, update the number of user-added cards: <pre>you_score.words = int(you_score.words) + number steps_txt2.words = int(steps_txt2.words) + 1</pre> Card must be displayed for a short time and then hidden. Use async... await... construct <pre>await play.timer(speed) card.hide()</pre> As per rules, a check whether the sum has become greater than 21 must be performed each time user adds a card. If the score count occurred to exceed 21, the player automatically loses. Take this into account:	Individual work, work in pairs, work with a teacher. *You can assist a student in working with files (address a card image file)	Button click events are handled in the program. The program can not yet determine loser and winner.
---	--	---	--	---

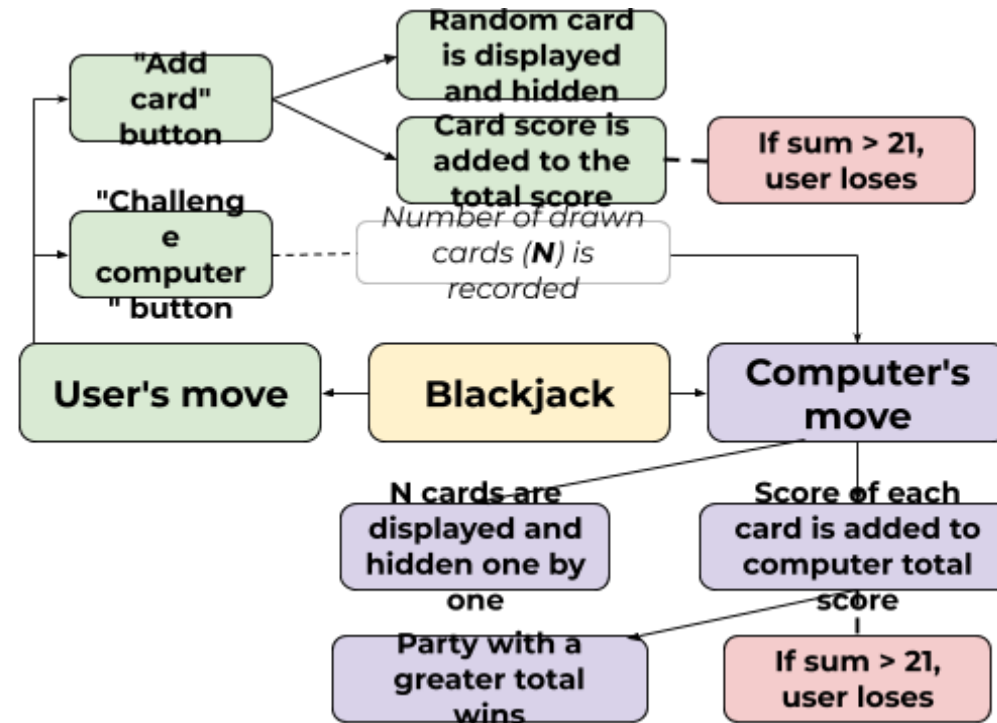
		<pre> # check the score: if int(you_score.words) > 21: # bust! lose() #display defeat </pre> Step 2. "Challenge computer" button press (computer's move). Similarly, computer draws random cards from its set and increases the score counter. The only difficulty is that computer must draw as many cards as user had -> card drawing may be performed in a loop executed a known number of times (this information is stored in steps_txt2). <pre> card.x = 150 # computer cards on the right steps = int(steps_txt2.words) # number of completed moves is displayed on the screen for i in range(steps): # do the same number of times number = randint(1, 11) ... </pre>		
6	Win and lose situations	Stage is set for handling win and lose situations. This situations are described as follows: ❑ User wins if computer score is greater than 21 (automatic win) or less than user score. It is convenient to check the following conditions in @button_stop.when_clicked. Then it will be done automatically when computer draws the required cards.	Individual work, work in pairs. *At this stage, no pair debate is intended. However, do not prevent one student from	The program is complete. You need to test it and deliver the project to customer.

		❑ User loses if previous condition is not met and his/her score is less than that of computer. ❑ Otherwise, draw is declared (scores are equal). <pre># win and lose if int(comp_score.words) > 21 or int(comp_score.words) < int(you_score.words): win() elif int(comp_score.words) > int(you_score.words): lose() else: draw()</pre> See the complete possible solution below.	helping another.	
--	--	---	------------------	--

7	Project delivery (testing by teacher)	Once a student declares project completion, arrange testing. It is performed in three stages: - Testing by project author. Student must open the project description and check if required actions are performed correctly. - Mutual project testing by students. Performed similarly to code review. - Project testing by teacher. Check the program. If everything works correctly, ask student comprehension questions on the topic. If student answers those successfully, offer additional tasks.	Individual work, work in pairs, work with a teacher.	Basic part of the project is complete. If there is enough time, student starts doing additional tasks.
---	--	---	--	--

Part 4. Sample project implementation

Project workflow:



Code

```
import play
from random import randint

# game rules:
hello = play.new_text(words='Try to get 21 points and beat the comp!', x=0, y=200)
```

```

# buttons:
button_add = play.new_box(color='yellow', border_width=1, border_color='grey', x=-130, y=-160, width=170, height=40)
add_txt = play.new_text(words='Add card', font_size=30, x=-130, y=-160, color='grey')

button_stop = play.new_box(color='red', border_width=1, border_color='grey', x=130, y=-160, width=190, height=40)
stop_txt = play.new_text(words='Start me VS comp', font_size=30, x=130, y=-160, color='grey')

# card:
card = play.new_image(image='card1.png', size=40, x=-150, y=20)
card.hide()

# results:
you_score = play.new_text(words='0', x = -100, y=-230)
comp_score = play.new_text(words='0', x = 100, y=-230)
result = play.new_text(words='', x=-200, y=-230)
steps_txt1 = play.new_text(words='Made steps: ', x=-20, y=-280, font_size=30)
steps_txt2 = play.new_text(words='0', x=60, y=-280, font_size=30) # number of user moves

speed = 1.5 # pause between moves (sec)

def lose():
    comp_score.color='green'
    you_score.color='red'
    result.color = 'red'
    result.words = 'You lose: ' # defeat

def win():
    comp_score.color='red'
    you_score.color='green'

```

```

result.color = 'green'
result.words = 'You win: ' # winning

def draw():
    result.words = 'Tie game ' # draw

@button_add.when_clicked
async def add_card():
    number=randint(1,11) # pick a random number
    # show a card with this number and add this score to the counter:
    card.image = 'card' + str(number) + '.png'
    card.show()
    you_score.words = int(you_score.words) + number
    steps_txt2.words = int(steps_txt2.words) + 1
    await play.timer(speed)
    card.hide()
    # check the score:
    if int(you_score.words) > 21: # bust! show defeat
        lose()

@button_stop.when_clicked
async def comp_play():
    card.x = 150 # mputer cards on the right
    steps = int(steps_txt2.words) # number of completed moves is displayed on the screen
    for i in range(steps): # do the same number of times
        number = randint(1, 11)
        card.image = 'card' + str(number) + '.png'
        card.show()
        comp_score.words = int(comp_score.words) + number

```

```

    await play.timer(speed)
    card.hide()

# win and lose
if int(comp_score.words) > 21 or int(comp_score.words) < int(you_score.words):
    win()
elif int(comp_score.words) > int(you_score.words):
    lose()
else:
    draw()

play.start_program()

```



Part 5. Result evaluation methods

Stage 1. *Program operability check*

Any program must handle the following test cases. If the test is passed, student is considered to achieve mandatory comprehension level.

Program testing

- | | |
|---|--|
| <ul style="list-style-type: none"> • Program is started, nothing pressed | <ul style="list-style-type: none"> • Nothing happens. |
| <ul style="list-style-type: none"> • "Add card" button is pressed | <ul style="list-style-type: none"> • A random card is displayed and hidden. The card score is added to user total score. Counter of drawn cards is incremented. |
| <ul style="list-style-type: none"> • "Challenge computer" button is pressed. | <ul style="list-style-type: none"> • Computer starts to play against user: each is dealt a random card, total scores are counted. |

Program notifies user whether he/she has won or lost as soon as game result becomes available.

Stage 2. *Student knowledge level inspection.*

Note. Teacher uses this section not to evaluate student publicly but personally assess his/her level of knowledge in the group.

There were no new advanced solutions in Blackjack project. Rather, this program summarizes knowledge gained in the project module. This is why a detailed knowledge check is not required at this stage. However, you can ask one of the following questions.

<i>Student knowledge inspection</i>	
What happens if "Add card" button is pressed after computer's move? Why?	Another random card will be dealt to user, total score will be updated. To put it simply, end of the game (hiding or blocking sprites) has not yet been programmed.
Recap sprite types you used in this program. Which methods did you use to work with sprites?	Block sprites, text sprites, image sprites. <code>hide()</code> and <code>show()</code> methods were applied to hide and display a sprite. Also, sprite properties such as position coordinates, size, shape, etc. were used.
How did you handle events in your project?	Object clicking event was handled in a special section. For example, response actions for <code>button_stop</code> click were described in <code>@button_stop.when_clicked</code> section.
Program contains the following commands: <pre>...int(you_score.words) < 22...</pre> <pre>comp_score.words = int(comp_score.words)+(number+1)</pre> Why changing type to numerical is only done in	- In the first case, we compared user score (which is a number) against 22. Type must definitely be numerical, whereas <code>you_score.words</code> is a string -> a conversion is required. - In the second case, we need to display new computer score on the screen. We do not need to do any arithmetics, so type conversion is not required.

<i>the first line, although the second one also contains numbers?</i>	
---	--



Part 6. Additional tasks for project improvement

Students wishing to improve their project can be offered additional tasks.

Task 1. Your game will become even more interesting if you make card drawing more realistic. To do this, eliminate the possibility of a card being drawn more than once. This is what happens in real life: player draws a card which stays with him/her and can not be drawn again.

Hint. To avoid card duplication, put all cards in `cards = []` and delete drawn cards from it one by one. We have programmed a similar construct in Arcanoid project.

Task 2. Program a visual design for the end of the game. What happens if user loses — will all sprites be hidden or "fall down"? Will clicking on buttons be blocked? Will sprites start flashing if user wins?

Sample solution for additional tasks 1 and 2 is provided [here](#).

Task 3. Set the background music! Download a music file from the Internet and add it to the project in "Add file" tab. Then load Pygame, add music to the program and enable it in `@play.when_program_start` section.

BLACKJACK

#Easy #Individual #Expectation_theory

Dear developer, you have been assigned an order for **BLACKJACK** program.

THE PROJECT must:

- Display the following UI elements on game startup: hints, "Add card" and "Challenge computer" buttons, score counters and number of drawn cards.
- Generate random cards for the user on pressing "Add card". If the score occurs to exceed 21, user loses.
- When "Challenge computer" button is pressed, computer makes a move: random cards are drawn and scores are added up in a similar fashion. This results in two scores, one for user and one for computer. If score occurs to exceed 21, computer loses.

User loses if his/her score is less than computer's or exceeds 21.

User wins if his/her score is more than computer's and less than 22.

It is recommended to implement this program in Python **over 2 lessons**.

Testing is a critical software development stage. It consists of 3 stages:

- *Independent program operability check*. For example, you need to check if:
 - Does counter value increase when a new random card is drawn?
 - Is the number of drawn cards counted when user makes a move?
 - Is winner decided correctly?
- *Program peer review by a developer*.

After successful testing, the project is ready for presenting to the customer. This is the final stage of testing the project.

